

ASP.NET Core Web API and HTML/CSS/JavaScript Frontend — Technology Stack

ASP.NET Core Web API

1. ASP.NET Core Web API

- **Purpose:** Core framework to build RESTful APIs.
- **Library:** Microsoft.AspNetCore.WebApi

2. Entity Framework Core

- **Purpose:** ORM for database operations using code-first approach.
- **Library:** Microsoft.EntityFrameworkCore

3. Swagger / Swashbuckle

- **Purpose:** To generate API documentation and allow API testing during development.
- **Library:** Swashbuckle.AspNetCore

4. AutoMapper

- **Purpose:** For mapping between DTOs (Data Transfer Objects) and Entities.
- **Library:** AutoMapper

5. FluentValidation

- **Purpose:** For clean and maintainable model validation.
- **Library:** FluentValidation.AspNetCore

6. JWT Authentication (JSON Web Token)

- **Purpose:** Stateless authentication for securing APIs.
- **Library:** Microsoft.AspNetCore.Authentication.JwtBearer

7. CORS (Cross-Origin Resource Sharing)

- **Purpose:** To allow HTML/JavaScript frontend to call the API hosted on a different domain/port.
- **Library:** Microsoft.AspNetCore.Cors

8. Serilog

- **Purpose:** Structured logging and error tracking.
- **Library:** Serilog.AspNetCore

9. Response Caching

- **Purpose:** Improve performance by caching GET API responses.

- **Library:** Microsoft.AspNetCore.ResponseCaching

10. Background Services

- **Purpose:** For background operations like scheduled tasks or periodic updates.
 - **Library:** Microsoft.Extensions.Hosting
-

HTML, CSS, JavaScript (Frontend)

1. HTML5

- **Purpose:** Markup language for structuring the frontend pages.
- **Technology:** Standard HTML5

2. CSS3

- **Purpose:** Styling and layout of the frontend pages.
- **Technology:** Standard CSS3

3. JavaScript (Vanilla JS)

- **Purpose:** To handle dynamic frontend logic, interact with backend APIs using fetch, and update the UI.
- **Technology:** Native JavaScript

4. Fetch API

- **Purpose:** For making asynchronous HTTP requests to the Web API (GET, POST, PUT, DELETE).
- **Technology:** Browser's built-in Fetch API

5. Bootstrap (optional)

- **Purpose:** To use pre-designed UI components and responsive layouts.
- **Library:** Bootstrap 5

6. SweetAlert2 / Toastr (optional)

- **Purpose:** For user-friendly alerts, confirmation modals, and notifications.
- **Libraries:** sweetalert2, toastr

7. SessionStorage / LocalStorage

- **Purpose:** To store JWT tokens securely on the frontend after login for authentication.
- **Technology:** Web Storage API (native browser)

Full-Stack Integration

1. CORS (Cross-Origin Resource Sharing)

- **Purpose:** To allow HTML/JavaScript frontend to securely interact with ASP.NET Core Web API backend.
- **Setup:** Configured inside Program.cs (or previously Startup.cs) using .UseCors() middleware.

2. JWT Authentication

- **Purpose:** Backend issues a JWT after successful login, which frontend stores in sessionStorage or localStorage.
- **Libraries:**
 - Backend: Microsoft.AspNetCore.Authentication.JwtBearer
 - Frontend: Native fetch API with Authorization: Bearer <token> in headers